

Procédure technique

Capture, stockage, analyse de stratégies et backtests de graphiques TradingView

Système	Trading Station v2.0
Technologies	HTML5, JavaScript, Node.js, Express, Puppeteer, base de données
But	Créer une interface capable de capturer des graphiques, d'enregistrer les images et de les analyser ensuite comme supports de stratégie et de backtest.

Ce document synthétise les solutions possibles pour capturer des screenshots TradingView, les enregistrer localement ou en base de données, puis les exploiter dans une interface d'analyse de stratégies.

1. Objectifs du système

- Permettre une capture manuelle et instantanée d'un graphique depuis une interface HTML.
- Enregistrer les captures dans un dossier local ou via une référence stockée en base de données.
- Préparer une automatisation en arrière-plan avec Node.js et Puppeteer.
- Construire une interface de revue pour annoter les captures, comparer les configurations et suivre les résultats.
- Obtenir un journal visuel de trading pour analyser les stratégies et les backtests.

2. Solutions de capture disponibles

Trois approches sont possibles selon le niveau de contrôle souhaité et la bibliothèque utilisée.

Solution	Usage	Avantage principal	Limite
Advanced Charts	Graphiques TradingView avancés	Méthode native <code>widget.takeClientScreenshot()</code>	Accès soumis à demande
Lightweight Charts	Interface HTML personnalisée	Simple, gratuite, rapide	Fonctions plus limitées
Puppeteer	Capture automatique côté serveur	Robuste pour les captures planifiées	Demande une configuration serveur

2.1 Solution officielle : Advanced Charts

Avec la Charting Library avancée de TradingView, la méthode native `widget.takeClientScreenshot()` permet de générer un canvas HTML5 du graphique courant.

```
async function captureChart() {
  const canvas = await widget.takeClientScreenshot();
  const dataURL = canvas.toDataURL('image/png');

  await fetch('/upload-screenshot', {
    method: 'POST',
    body: JSON.stringify({ image: dataURL })
  });
}
```

```

    headers: { 'Content-Type': 'application/json' }
  });
}

```

2.2 Solution open source : Lightweight Charts

La bibliothèque Lightweight Charts rend le graphique dans un élément canvas. La méthode `chart.takeScreenshot()` retourne une image exploitable en Base64 ou en Blob.

2.3 Solution automatisée : Node.js + Puppeteer

Puppeteer ouvre une instance invisible de Chrome, charge la page contenant le graphique, attend le rendu des indicateurs, puis enregistre la capture.

```

const puppeteer = require('puppeteer');

(async () => {
  const browser = await puppeteer.launch();
  const page = await browser.newPage();
  await page.goto('VOTRE_URL_AVEC_GRAPHIQUE');

  await page.waitForSelector('.tv-lightweight-charts, iframe');

  await page.screenshot({ path: 'screenshot_analyse.png' });
  await browser.close();
  console.log("Capture sauvegardée !");
})();

```

3. Interface HTML de capture interactive

La première étape consiste à créer une page HTML affichant le graphique et un bouton de capture. Cette approche permet de valider le rendu visuel avant d'automatiser le processus.

```

<!DOCTYPE html>
<html>
<head>
  <title>TradingView Capture</title>
  <script src="https://unpkg.com/lightweight-charts/dist/lightweight-charts.standalone.
production.js"></script>
</head>
<body>
  <div id="chart-container" style="width: 600px; height: 400px;"></div>
  <button onclick="saveScreenshot()">Capturer et Enregistrer</button>

  <script>
    const chart = LightweightCharts.createChart(
      document.getElementById('chart-container'),
      { width: 600, height: 400 }
    );
    const lineSeries = chart.addLineSeries();
    lineSeries.setData([
      { time: '2023-01-01', value: 100 },
      { time: '2023-01-02', value: 120 }
    ]);

    async function saveScreenshot() {
      const canvas = chart.takeScreenshot();
      const dataURL = canvas.toDataURL('image/png');

      const response = await fetch('http://localhost:3000/upload', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          image: dataURL,
          name: "screenshot_" + Date.now()
        })
      });

      if (response.ok) alert("Image enregistrée côté serveur !");
    }
  </script>
</body>
</html>

```

4. Serveur Node.js pour le stockage local

Le serveur reçoit l'image encodée en Base64, retire l'en-tête de la donnée, puis écrit le fichier PNG dans un dossier local.

```
const express = require('express');
const fs = require('fs');
const app = express();

app.use(express.json({ limit: '10mb' }));

app.post('/upload', (req, res) => {
  const { image, name } = req.body;
  const base64Data = image.replace(/^data:image\/png;base64/, "");

  fs.writeFile(`./screenshots/${name}.png`, base64Data, 'base64', (err) => {
    if (err) return res.status(500).send(err);
    res.send("Fichier sauvegardé.");
  });
});

app.listen(3000, () => console.log('Serveur actif sur port 3000'));
```

Point important : le dossier /screenshots doit exister avant l'écriture, ou être créé automatiquement par le serveur.

5. Automatisation en arrière-plan

Une fois la capture manuelle validée, le même principe peut être automatisé. Le script Node.js lance un navigateur invisible, ouvre la page, attend le chargement du graphique et sauvegarde l'image.

Phase	Action
Installation	Installer Puppeteer avec <code>npm install puppeteer</code> .
Initialisation	Lancer Chromium en mode headless.
Navigation	Ouvrir l'URL du graphique ou de la page Trading Station.
Attente	Prévoir 3 à 5 secondes ou attendre un sélecteur précis.
Export	Utiliser <code>page.screenshot({ path: '...' })</code> .
Planification	Utiliser node-cron, PM2, Cron ou le Planificateur de tâches Windows.

6. Interface d'analyse des stratégies

L'analyse ne doit pas se limiter à l'image. Chaque capture doit être liée à des métadonnées afin de comprendre le contexte du trade ou du backtest.

Champ	Rôle
image_path	Lien vers le fichier image.
timestamp	Date et heure de la capture.
symbol	Actif analysé, par exemple BTCUSDT.
timeframe	Unité de temps, par exemple 1H ou 15m.
strategy_tag	Nom ou type de stratégie testée.

outcome	Résultat : gagnant, perdant, neutre ou en attente.
notes	Observations sur le price action et les signaux.
tags	Mots-clés : divergence, breakout, support, résistance, etc.

6.1 Structure de la page d'analyse

- Une colonne latérale pour lister les captures disponibles.
- Une zone centrale pour afficher l'image en grand format.
- Un module d'annotation avec résultat, notes et tags.
- Des filtres par actif, intervalle, stratégie, date ou résultat.
- Une option de comparaison côte à côte entre un setup initial et son résultat.

```
<div class="analysis-container">
  <aside class="sidebar">
    <input type="text" placeholder="Filtrer par symbole...">
    <div id="image-list">
      <!-- Liste des miniatures générée par JavaScript -->
    </div>
  </aside>

  <main class="viewer">
    

    <div class="annotation-box">
      <h3>Annotation du Setup</h3>
      <select id="outcome">
        <option value="pending">En attente</option>
        <option value="win">Gagnant</option>
        <option value="loss">Perdant</option>
      </select>
      <textarea id="notes" placeholder="Pourquoi ce screenshot ?"></textarea>
      <button onclick="updateAnalysis()">Sauvegarder l'analyse</button>
    </div>
  </main>
</div>
```

7. Backtests et statistiques

L'interface d'analyse peut devenir un journal de backtest visuel. Les captures annotées permettent de calculer des statistiques et d'identifier les configurations qui fonctionnent le mieux.

- Taux de réussite global : nombre de captures gagnantes divisé par le nombre total de captures analysées.
- Taux de réussite par stratégie : comparaison des setups, par exemple RSI, cassure, divergence ou moyenne mobile.
- Filtrage par actif : BTCUSD, ETHUSD ou autre actif suivi.
- Filtrage par intervalle : 5 minutes, 15 minutes, 1 heure, 4 heures, etc.
- Comparaison visuelle : setup avant trade et résultat après plusieurs bougies.

8. Recommandations de stockage

Élément	Méthode recommandée
Images	Dossier local /captures ou stockage externe comme S3 ou Cloudinary.
Base de données	Stocker le chemin ou l'URL de l'image, pas le fichier binaire lui-même.
Métadonnées	Fichier JSON, SQLite, PostgreSQL ou MySQL selon l'ampleur du projet.
Nom des fichiers	YYYY-MM-DD_HH-MM_SYMBOLE_INTERVALLE_STRATEGIE.png.
Tags	Prévoir des mots-clés pour retrouver rapidement les setups similaires.

Recommandation : éviter de stocker directement les images lourdes dans PostgreSQL ou MySQL. Il est préférable de stocker les images sous forme de fichiers, puis d'enregistrer uniquement leur chemin dans la base.

9. Analyse par intelligence artificielle

Une évolution possible consiste à envoyer les captures à un modèle capable d'analyser les images. L'IA peut préremplir les notes, repérer une divergence RSI, lire une structure de marché ou vérifier la distance entre le prix et une moyenne mobile.

Exemple de demande : Voici une capture d'écran de TradingView. Identifie si une divergence RSI est présente et décris la structure du marché visible.

10. Ordre de mise en œuvre conseillé

Étape	Travail à réaliser	Résultat attendu
1	Créer la page HTML avec graphique et bouton de capture.	Capture manuelle fonctionnelle.
2	Créer la route Node.js /upload.	Image enregistrée dans /screenshots.
3	Ajouter les métadonnées associées à chaque capture.	Chaque image a un contexte exploitable.
4	Créer la galerie d'analyse.	Consultation et annotation des captures.
5	Ajouter filtres et statistiques.	Suivi des performances par stratégie.
6	Automatiser avec Puppeteer.	Captures régulières sans intervention.
7	Prévoir une analyse IA optionnelle.	Aide à l'interprétation des captures.

Conclusion

La meilleure approche consiste à commencer par une interface HTML simple avec capture manuelle, puis à ajouter le serveur Node.js, les métadonnées, la galerie d'analyse et enfin l'automatisation. Cette progression réduit les erreurs et permet de valider chaque bloc avant de passer au suivant.

Le système final devient une base solide pour un journal de trading visuel, l'analyse de stratégies et la constitution d'un historique exploitable pour les backtests.